

Machine Learning for Data Analytics

Group Project — Getting Started Guide

Janusz Szwabiński

Table of Contents

1. [Project Requirements at a Glance](#)
2. [Grading Criteria](#)
3. [Step 0 — Form Your Team](#)
4. [Step 1 — Set Up Your Tools](#)
5. [Step 2 — Choose Your Project](#)
6. [Step 3 — Plan Your Work](#)
7. [Step 4 — Code Like Professionals](#)
8. [Step 5 — Track Your Experiments](#)
9. [Step 6 — Interpret Your Models](#)
10. [Step 7 — Prepare the Presentation](#)
11. [Common Mistakes to Avoid](#)
12. [Team Health Checklist](#)

1. Project Requirements at a Glance

Your project must include **all** of the following:

Requirement	Details
 Advanced techniques	Methods from different course modules
 Model interpretability	SHAP, LIME, or similar
 MLOps best practices	Experiment tracking, reproducibility
 Jupyter Notebook	Clean, documented code + findings
 Presentation	15–20 min (+ live demo, if applicable), slides as PDF
 GitHub Repository	Full version control history
 Experiment Tracking	MLflow or Weights & Biases
 Deadline	All materials 48 hours before presentation

2. Grading Criteria

Detaild grading criteria can be found at <https://eportal.pwr.edu.pl/course/view.php?id=5012>

3. Step 0 — Form Your Team

Team size: 3–4 people

Before anything else, decide on roles. You don't need rigid job titles, but everyone should know what they're primarily responsible for.

Suggested role distribution:

Role	Responsibilities
Team Lead	Coordinates tasks, tracks deadlines, facilitates decisions
Data Engineer	Data loading, cleaning, feature engineering
Modeler	Model design, training, hyperparameter tuning
Analyst / Communicator	Interpretability analysis, visualizations, presentation

💡 In a 3-person team, one member combines two roles. Decide this explicitly — don't assume!

📌 First team meeting agenda

- Exchange contact details
- Decide on a communication channel (Slack / Discord / Teams)
- Agree on a meeting cadence (e.g., weekly check-in)
- Assign initial roles
- Choose your project topic (see Step 2)

4. Step 1 — Set Up Your Tools

Set up your shared infrastructure **on Day 1**. Do not skip this.

GitHub Repository

One person creates the repo, everyone else forks or is added as a collaborator.

```
In [ ]: # Suggested repository structure
repo_structure = """
my-ml-project/
├── data/
│   ├── raw/           # Original, unmodified data
│   └── processed/      # Cleaned/transformed data
├── notebooks/
│   ├── 01_eda.ipynb
│   ├── 02_feature_engineering.ipynb
│   ├── 03_modeling.ipynb
│   ├── 04_interpretability.ipynb
│   └── 05_final_report.ipynb
├── src/
│   ├── data_loader.py
│   ├── features.py
│   └── models.py
├── mlruns/             # MLflow tracking (auto-generated)
├── reports/
│   └── presentation.pdf
├── requirements.txt
├── README.md
└── .gitignore
"""
print(repo_structure)
```

```
In [ ]: # Minimal .gitignore for ML projects
gitignore_content = """
# Data (often too large for GitHub)
```

```

data/raw/
data/processed/

# Python
__pycache__/*
*.pyc
.env

# Jupyter checkpoints
.ipynb_checkpoints/

# Model artifacts (can be large)
*.pkl
*.h5

# OS files
.DS_Store
Thumbs.db
"""
print(gitignore_content)

```

Shared Environment

Everyone must use the **same Python environment**. Use `requirements.txt` or `environment.yml`.

```

In [ ]: # Install core dependencies (run once per team member)
        # !pip install -r requirements.txt

        # Typical requirements.txt for this course:
        requirements = """
        scikit-learn>=1.3
        xgboost>=2.0
        lightgbm>=4.0
        shap>=0.44
        lime>=0.2
        mlflow>=2.9
        pandas>=2.0
        numpy>=1.26
        matplotlib>=3.8
        seaborn>=0.13
        torch>=2.1           # if using deep learning
        wandb>=0.16          # alternative to MLflow
        """
        print(requirements)

```

Git Workflow — Simple Branching Strategy

```

main          ← stable, presentation-ready code only
└─ develop    ← integration branch, merge features here
└─ first
    └─ feature/eda
    └─ feature/xgboost-model

```

Rules:

- Never commit directly to `main`
- Each team member works on their own feature branch
- Use Pull Requests to merge — this creates a review history
- Write meaningful commit messages: "Add SHAP summary plot for churn model" not "update"

5. Step 2 — Choose Your Project

Available projects are listed in the course materials. When choosing, consider:

Factor	Questions to ask
Interest	Does the problem genuinely interest your team?
Data availability	Is the dataset accessible and of reasonable quality?
Scope	Can you realistically complete it in the given time?
Technique coverage	Does it naturally lend itself to multiple advanced techniques?
Interpretability	Can you meaningfully apply SHAP/LIME?

 **Tip:** A well-executed simple project beats an ambitious project done poorly. Choose a scope you can finish thoroughly.

6. Step 3 — Plan Your Work

Suggested Timeline

Adapt this to your actual deadline, but keep the proportions.

Week	Milestone
3	Team formed, repo created, project chosen, data acquired
5	EDA complete, data cleaned, initial features ready
7	Baseline model working, experiment tracking active
9	Advanced models implemented (≥ 3 techniques)
11	Interpretability analysis (SHAP/LIME) complete

Week	Milestone
13	Notebook polished, presentation drafted
15	Full rehearsal, final submission (48h before presentation)

✓ Task Management

Use GitHub Issues, Trello, or a simple shared spreadsheet. Each task should have:

- A clear description
- An owner (one person is responsible)
- A due date
- A status (To Do / In Progress / Done)

7. Step 4 — Code Like Professionals

Your Jupyter notebook is worth **15% of your grade**. Quality matters.

🔑 Notebook Best Practices

```
In [ ]: # ✓ GOOD: Clear imports at the top, with comments
import pandas as pd          # data manipulation
import numpy as np           # numerical computing
import matplotlib.pyplot as plt # visualization
import shap                  # model interpretability
import mlflow                 # experiment tracking

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import classification_report, roc_auc_score
from xgboost import XGBClassifier

# Reproducibility – always set a random seed!
RANDOM_SEED = 42
np.random.seed(RANDOM_SEED)
```

```
In [ ]: # ✓ GOOD: Document what you're doing with markdown cells AND inline comment

def load_and_validate_data(filepath: str) -> pd.DataFrame:
    """
    Load dataset and perform basic validation.

    Parameters
    -----
    filepath : str
        Path to the CSV file.

    Returns
    -----
```

```

pd.DataFrame
    Validated DataFrame ready for further processing.
"""
df = pd.read_csv(filepath)

# Basic sanity checks
assert not df.empty, "Dataset is empty!"
print(f"✅ Loaded {len(df):,} rows and {df.shape[1]} columns")
print(f"Missing values:\n{df.isnull().sum()[df.isnull().sum() > 0]}")

return df

# Example usage (replace with your actual path)
# df = load_and_validate_data("data/raw/telco_churn.csv")

```

```

In [ ]: # ❌ BAD – don't do this:
# df2 = df.dropna() # no explanation
# model = XGBClassifier() # magic numbers, no documentation
# model.fit(X, y)

# ✅ GOOD – do this instead:
# Drop rows with missing target values (business requirement: can't impute c
# df_clean = df.dropna(subset=['churn'])
# print(f"Removed {len(df) - len(df_clean)} rows with missing churn labels")

```

Notebook Structure Checklist

Every final notebook should have these sections (in order):

1. **Introduction** — What problem are you solving? Why does it matter?
2. **Data Loading & Validation** — Where does the data come from? What are its characteristics?
3. **Exploratory Data Analysis (EDA)** — Visualize distributions, correlations, class imbalance
4. **Feature Engineering** — Transformations, encoding, new features
5. **Modeling** — Baseline → Advanced models (≥3 techniques)
6. **Evaluation** — Metrics, comparison table, confusion matrices
7. **Interpretability** — SHAP/LIME analysis, business insights
8. **Conclusions** — Key findings, limitations, future work

 Before submission: **Restart kernel and run all cells** to ensure everything works top-to-bottom!

8. Step 5 — Track Your Experiments

Experiment tracking is a **required** MLOps practice. Use **MLflow** or **Weights & Biases**.

MLflow Quick Start

```
In [ ]: import mlflow
import mlflow.sklearn
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score

# Start the MLflow UI with: mlflow ui (in terminal)
# Then open: http://localhost:5000

mlflow.set_experiment("churn-prediction") # name your experiment

with mlflow.start_run(run_name="random_forest_baseline"):

    # Log hyperparameters
    params = {"n_estimators": 100, "max_depth": 6, "random_state": RANDOM_SE
mlflow.log_params(params)

    # Train model
    model = RandomForestClassifier(**params)
    model.fit(X_train, y_train)

    # Log metrics
    auc = roc_auc_score(y_test, model.predict_proba(X_test)[:, 1])
    mlflow.log_metric("roc_auc", auc)
    mlflow.log_metric("accuracy", model.score(X_test, y_test))

    # Log the model itself
    mlflow.sklearn.log_model(model, "random_forest_model")

print("✅ Run logged to MLflow!")
print(f"    ROC-AUC: {auc:.4f}")
```



Why Track Experiments?

- **Reproducibility:** Anyone can re-run the exact same experiment
- **Comparison:** Easily compare 10 models and pick the best
- **Accountability:** Know *who* ran *what* and *when*
- **Professionalism:** This is standard industry practice

💡 Log **every** experiment, even failed ones. Negative results are valuable!

9. Step 6 — Interpret Your Models

Model interpretability is worth **20% of your grade**. Don't treat it as an afterthought.

SHAP Values — Quick Start


```
In [ ]: import shap

# After training your model (e.g., XGBoost):
# explainer = shap.TreeExplainer(model)
# shap_values = explainer.shap_values(X_test)

# --- Global explanation: what features matter most overall? ---
# shap.summary_plot(shap_values, X_test, plot_type="bar")

# --- Detailed summary: direction of impact ---
# shap.summary_plot(shap_values, X_test)

# --- Local explanation: why did THIS prediction happen? ---
# shap.waterfall_plot(shap.Explanation(shap_values[0], base_values=explainer

print("SHAP analysis template ready – replace commented code with your actua
```

What to Include in Your Interpretability Section

Analysis	What it shows	Required?
SHAP summary plot (bar)	Global feature importance	✓ Yes
SHAP beeswarm plot	Feature impact direction	✓ Yes
SHAP waterfall / force plot	Single prediction explanation	✓ Yes
SHAP dependence plot	Feature interaction effects	Recommended
Counterfactual explanations	"What would need to change?"	Bonus

 **Don't just show plots — interpret them!** Write 2–3 sentences explaining what each plot tells you in business terms.

10. Step 7 — Prepare the Presentation

Your presentation is **15–20 minutes + live demo**. Slides must be submitted as PDF.

Suggested Slide Structure

Slide	Content	Time
1	Title, team members, project name	30s
2	Problem statement & motivation	1–2 min
3	Dataset overview & EDA highlights	2 min
4	Methods used (≥3 techniques)	2–3 min

Slide	Content	Time
5	Results & model comparison	2 min
6–7	Interpretability analysis (SHAP)	3 min
8	Live demo — walk through the notebook	3–4 min
9	Conclusions & lessons learned	1–2 min
10	Q&A	remaining

Presentation Tips

- **Everyone must speak.** Assign slides to team members in advance.
- **Practice the live demo.** Kernel should be restarted and run *before* the presentation.
- **Anticipate questions:** "Why did you choose XGBoost over a neural network?", "What would you do with more data?"
- **Tell a story:** problem → data → approach → results → insights → impact.
- **Slides should be visual,** not walls of text. One idea per slide.

11. Common Mistakes to Avoid

Learn from others' mistakes before making your own. 😊

✗ Technical Pitfalls

Mistake	Why it's bad	Fix
Data leakage	Inflated metrics that won't generalize	Apply all preprocessing <i>inside</i> cross-validation
Not setting a random seed	Irreproducible results	Always set <code>random_state=42</code>
Reporting only accuracy	Misleading for imbalanced data	Use AUC-ROC, F1, precision/recall
Training on test data	Invalid evaluation	Strict train/val/test split from the start
No baseline model	Can't show improvement	Always start with a simple baseline (e.g., <code>DummyClassifier</code>)

✗ Team / Process Pitfalls

Mistake	Why it's bad	Fix
---------	--------------	-----

Mistake	Why it's bad	Fix
Starting too late	No time to iterate	Begin coding early
One person does everything	Others learn nothing; one person burns out	Use task tracking, hold each other accountable
No version control	Conflicts, lost work	Commit frequently, at least daily
Notebook won't run from scratch	Fails at evaluation	Restart kernel + run all, every time
Interpretability added last minute	Shallow analysis	Plan for interpretability from the start

12. Team Health Checklist

Use this checklist at your **weekly team meetings** to stay on track.

Week 3

- ☐ Team formed and roles assigned
- ☐ Communication channel set up
- ☐ GitHub repository created with correct structure
- ☐ Project topic chosen and confirmed with instructor
- ☐ Data acquired and placed in `data/raw/`
- ☐ `requirements.txt` committed and tested by all members

Mid-Project

- ☐ EDA notebook complete and pushed
- ☐ At least one model running end-to-end
- ☐ MLflow / W&B tracking active
- ☐ multiple advanced techniques implemented
- ☐ SHAP/LIME analysis started
- ☐ Everyone has made commits to the repository

Final Week (48h before presentation)

- ☐ Final notebook runs cleanly from top to bottom
- ☐ All results are logged in MLflow
- ☐ Interpretability section complete with written explanations
- ☐ Presentation slides finalized and exported as PDF
- ☐ Live demo rehearsed at least once
- ☐ GitHub repo is clean and README is up to date

- ☐ All materials submitted 48h before presentation ☒

Final Advice

Three things that separate great projects from average ones:

1. **Clarity of thought.** A well-explained simple model beats a confusing complex one. Always ask: *"Would a non-technical person understand what we're doing and why?"*
2. **Reproducibility.** If your project can't be run by someone else from scratch, it's not complete. Test this before submission.
3. **Team communication.** Most project failures are communication failures. Talk to each other early and often. Raise problems quickly — don't wait until they become crises.

"Coming together is a beginning, staying together is progress, and working together is success."

— Henry Ford

Good luck! 